Windows Detection Lab – Sigma Rules

Blue Team Windows Detection engineering study

Author: Oscar (OscarXMT) | Completed: 16/10/2026

Project overview

I recently thought how weird windows logging really is, I decided to test it out myself, so I started out with using chainsaw for the sigma rules and then I used Sysmon and enabled it to monitor the system. I also set up a PowerShell script that would write all script block logs to a file. After testing out with some suspicious commands (accessing the temp directory etc.). The sigma rules worked great, catching nearly all events.

Research context and scope

This simulation was of course completely harmless, all we did was test a couple of safe commands that would likely trigger our sigma rules (powershell -EncodedCommand), (schtasks /run /tn). We also tested executing a safe binary in the %TEMP% folder. The scope of this is not massive either attacks could simply not use these methods and paths for their attack chains, and our sigma rules would not detect them.

Lab setup

Component	Your configuration
Windows VM	Windows 11 Home (OS Build 26200.8037)
Resources and network	Ryzen 9 5900x, 10 Gigabytes of RAM, NAT Network
Logging	Sysmon v15.22 - Script blocking PowerShell logging
Detection tools	Chainsaw Version 2.17.1

Workflow:

Windows VM → harmless simulations → Sysmon → EVTX exports → Sigma and Chainsaw → detection results

Detection coverage and test results

Detection	Harmless simulation	Evidence to inspect
Scheduled task creation	Create and remove a harmless task	Process command line containing /create
Encoded PowerShell	Run an encoded command that prints text	PowerShell command line containing -EncodedCommand
Execution from Temp	Run a copy of lab-whoami.exe from Temp	The executable hiding in the Temp directory

Rule files

These files are present in the githubs repo in the rules folder with the pdf.

Scheduled task: scheduled_task_creation.yml

Encoded PowerShell: powershell_encoded_command.yml

Temp execution: execution_from_temp.yml

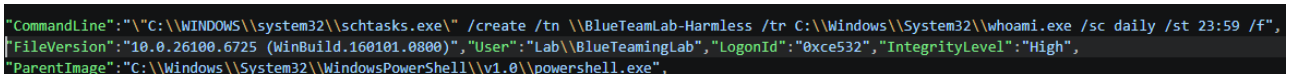
Validation method

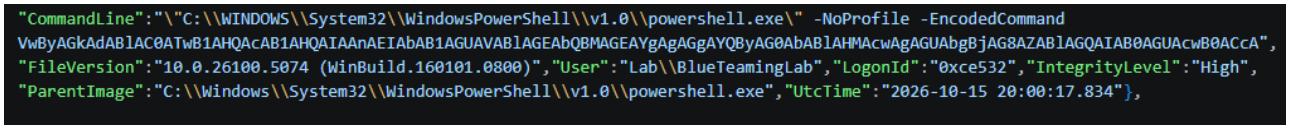
Detection	Positive test result	Found result	Notes
Scheduled task	The command has been tracked and was deemed suspicious by chainsaw	6 Detections have been found. Schtasks /run and Schtasks /create	Some unexpected ones were legitimate windows scheduled tasks like windows teams.
Encoded PowerShell	The sigma rule we set up caught the script block log immediately. NOTE the Mitre Att&ck id for this is “T1059.001”	Powershell -EncodedCommand “BASE64”	NOTE: Some legitimate programs/apps will use -EncodedCommand so false positives can be common.
Temp execution	We caught all the binaries running out of **the** %TEMP% folder. Even the ones that we weren’t testing.	%TEMP%/lab-whoami.exe has been detected for executing out of temp directory.	We also got a false positive when trying to later install git on the system the sigma rules immediately lit up.

Some Example screenshots:









Screenshots and evidence

We first run this script to start “Script Block Logging”

```
script-block.ps1
1  $loggingPath = 'HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging'
2
3  New-Item -Path $loggingPath -Force | Out-Null
4
5  New-ItemProperty -Path $loggingPath `
6      -Name EnableScriptBlockLogging `
7      -PropertyType DWord `
8      -Value 1 `
9      -Force | Out-Null
```

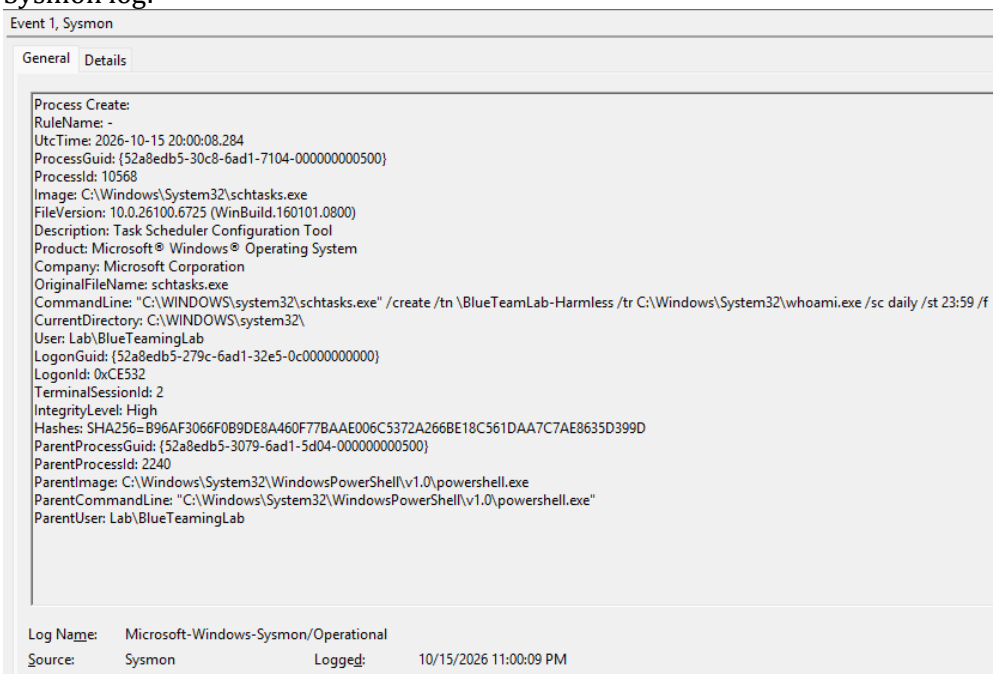
We also run Sysmon to start monitoring. By running this command with administrator privileges.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> .\Sysmon64.exe -accepteula -i
```

Scheduled task creation

Sysmon log:



Chainsaw Detections:


```
"CommandLine":"\"C:\\WINDOWS\\system32\\schtasks.exe\" /create /tn \\BlueTeamLab-Harmless /tr C:\\Windows\\System32\\whoami.exe /sc daily /st 23:59 /f",
"FileVersion":"10.0.26100.6725 (WinBuild.160101.0800)","User":"Lab\\BlueTeamingLab","LogonId":"0xce532","IntegrityLevel":"High",
"ParentImage":"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
```


The evidence we see here proves that the sigma rules chainsaw parsed through the Sysmon logs, did in fact catch the command being executed correctly/as intended. But a problem with that is of course that very real apps/programs also use it for startup so false detections would be common we can of course attempt to create filters in the sigma rules like only detect unsigned binaries/scripts.

The commands ran to trigger this Sysmon event:

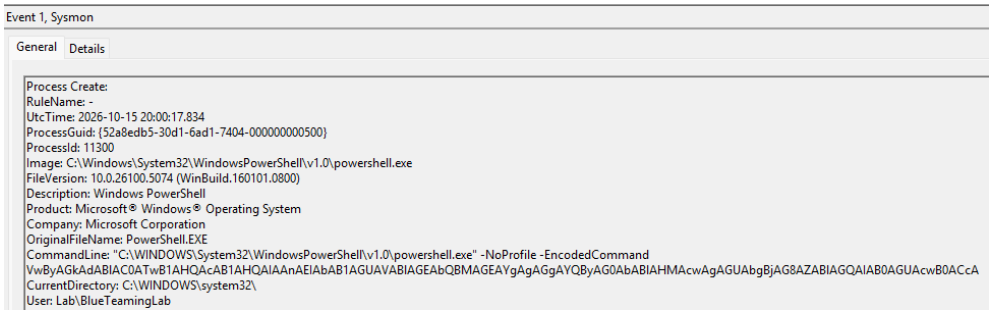
```
CREATE_SCHTASKS.ps1
1-Tests > CREATE_SCHTASKS.ps1
1  schtasks.exe /create /tn "\BlueTeamLab-Harmless" /tr "C:\Windows\System32\whoami.exe" /sc daily /st 23:59 /
2
3  schtasks.exe /query /tn "\BlueTeamLab-Harmless"
4
5  schtasks.exe /delete /tn "\BlueTeamLab-Harmless" /f
```

The sigma rule we wrote for this detection:

```
rules > scheduled_task_creation.yml
1  title: Scheduled Task Creation Via Schtasks
2  id: 76de84bf-840d-4d97-b6b2-8aa3fed4ea68
3  status: experimental
4  description: Detects schtasks being invoked to create a task; does not prove creation succeeded.
5  author: Blue Team Lab
6  date: 2026-10-15
7  tags:
8      - attack.execution
9      - attack.persistence
10     - attack.t1053.005
11 logsource:
12     category: process_creation
13     product: windows
14 detection:
15     selection:
16         Image|endswith: '\schtasks.exe'
17         CommandLine|contains: '/create'
18     condition: selection
19 falsepositives:
20     - Legitimate administrator and software installation activity
21 level: low
```

We even added the Mitre Att&ck ID (T1053.005) for this sigma rule.

Encoded PowerShell



Chainsaw command:


```
"CommandLine":"\"C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe\" -NoProfile -EncodedCommand
VwByAGkAdABlAC0ATwB1AHQAcAB1AHQAIAAnAEIAbAB1AGUAVABlAGEAbQBMAGEAYgAgAGgAYQByAG0AbABlAHMAcwAgAGUAbgBjAG8AZABlAGQAIAB0AGUAcwB0ACcA",
"FileVersion":"10.0.26100.5074 (WinBuild.160101.0800)","User":"Lab\\BlueTeamingLab","LogonId":"0xce532","IntegrityLevel":"High",
"ParentImage":"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe","UtcTime":"2026-10-15 20:00:17.834"},
```


We can see here that the sigma rules we created successfully caught the encoded PowerShell command! This proves that the sigma rules will filter inside of the Sysmon logs for any PowerShell commands with -EncodedCommand parameter.

But keep in mind some apps/programs can use this for legitimate purposes so false positives cannot be ruled out!

Bad actor command (triggering Sysmon logging):

```
PS C:\WINDOWS\system32> powershell -NoProfile -EncodedCommand VwByAGkAdABlAC0ATwB1AHQAcAB1AHQAIAAnAEIAbAB1AGUAVABlAGEAbQBMAGEAYgAgAGgAYQByAG0AbABlAHMAcwAgAGUAbgBjAG8AZABlAGQAIAB0AGUAcwB0ACcA
BlueTeamLab harmless encoded test
PS C:\WINDOWS\system32>
```

The sigma rule we wrote for this detection:

```yaml
title: PowerShell EncodedCommand Parameter
id: cbb398cc-ed3a-4b31-b921-a0edb91967fb
status: experimental
description: Detects the full EncodedCommand parameter in a PowerShell command line.
author: Blue Team Lab
date: 2026-10-15
tags:
    - attack.execution
    - attack.t1059.001
logsource:
    category: process_creation
    product: windows
detection:
    selection:
        Image|endswith:
            - '\powershell.exe'
            - '\pwsh.exe'
        CommandLine|contains: ' -EncodedCommand '
    condition: selection
falsepositives:
    - Legitimate automation using encoded commands
level: medium
```

Execution from Temp

Event 1, Sysmon

General Details

```
Process Create:
RuleName: -
UtcTime: 2026-10-15 20:00:24.046
ProcessGuid: {52a8edb5-30d8-6ad1-7504-000000000500}
ProcessId: 9472
Image: C:\Users\BLUETE~1\AppData\Local\Temp\BlueTeamLab\lab-whoami.exe
FileVersion: 10.0.26100.1882 (WinBuild.160101.0800)
Description: whoami - displays logged on user information
Product: Microsoft® Windows® Operating System
Company: Microsoft Corporation
OriginalFileName: whoami.exe
CommandLine: "C:\Users\BLUETE~1\AppData\Local\Temp\BlueTeamLab\lab-whoami.exe"
CurrentDirectory: C:\WINDOWS\system32\
User: Lab\BlueTeamingLab
LogonGuid: {52a8edb5-279c-6ad1-32e5-0c0000000000}
LogonId: 0xCE532
TerminalSessionId: 2
IntegrityLevel: High
Hashes: SHA256=23240EF9F8B0A9A324110B1C2331DE31DC1B0E08F5359CB707E51A939AF56CD3
ParentProcessGuid: {52a8edb5-3079-6ad1-5d04-000000000500}
```

Chainsaw detecting it:


```
"CommandLine":"\"C:\\Users\\BLUETE~1\\AppData\\Local\\Temp\\BlueTeamLab\\lab-whoami.exe\"",
"FileVersion":"10.0.26100.1882 (WinBuild.160101.0800)",
"User":"Lab\\BlueTeamingLab",
"LogonId":"0xce532",
"IntegrityLevel":"High",
"ParentImage":"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
```


Chainsaw getting false positive


```
:{"Image":"C:\\Users\\BLUETE~1\\AppData\\Local\\Temp\\is-NV16WHJQY2.tmp\\Git-2.56.0.2-64-bit.tmp","RuleName":"-","Description":"Setup/Uninstall","OriginalFileName":"
"TerminalSessionId":2,"ParentCommandLine":"\"C:\\Users\\BlueTeamingLab\\Downloads\\Git-2.56.0.2-64-bit.exe\" ","Product":"Git
```


Figure 3.

We can see here that the sigma rules did indeed detect the “lab-whoami.exe” executing from “%TEMP%” which is a common place for malware to hide. But we did indeed get a false positive, for when running the real git installer for windows the sigma rule instantly flagged it. So false positives are going to be very common with this rule hence forth why we need to add a filter to this like if the binary is digitally signed or not.

Evidence reference:

The bad actor commands we ran.

```powershell
$testFolder = Join-Path $env:TEMP 'BlueTeamLab'
New-Item -ItemType Directory -Path $testFolder -Force | Out-Null

Copy-Item "$env:WINDIR\System32\whoami.exe" "$testFolder\lab-whoami.exe"

& "$testFolder\lab-whoami.exe"

Remove-Item -Path "$testFolder\lab-whoami.exe"
```

The sigma rule we wrote for TEMP execution:

```yaml
title: Process Execution From A Temp Directory
id: 29156d6e-d47c-43dd-9eca-68ac9dcf2813
status: experimental
description: Detects executables launched from a path containing a Temp directory.
author: Blue Team Lab
date: 2026-10-15
logsource:
    category: process_creation
    product: windows
detection:
    selection:
        Image|contains: '\Temp\'
    condition: selection
falsepositives:
    - Installers and software updaters
    - Legitimate portable applications
level: low
```

Detection limitations & lessons learned

- Scheduled task creation: a schtasks /create process event shows an invocation; it does not establish that the task was created successfully.
- Encoded PowerShell: a rule matching only the full -EncodedCommand spelling misses abbreviated forms like (-enc -eNcOdEDCoMmAnD).
- Temp execution: legitimate installers and updaters may execute from temporary folders and generate matches. Like in our case with the very real and legitimate git being flagged by the sigma rule.
- Coverage: Successful simulations validate the tested cases; they do not establish comprehensive malware detection.
- While sigma rules are powerful for very specific attack patterns, we need to filter for a lot of niche cases.

Troubleshooting and tuning

I expected the rules to catch most of the scenarios we threw at them, I also expected there to be some false positives especially with the execution from temp rule. When testing the rules most of these assumptions became true all the scenarios were detected by the rules and we did indeed hit a false positive. The only tuning that is going to be needed with sigma rules is adding filters for different attack patterns.

What I learned

I learned in depth how logging works in windows. Not only that, but how powerful sigma rules can be, if written correctly of course. ;)

Although the flexibility of how you write the sigma rules is very awesome (with filters etc).

Next improvements

Configuring Sysmon to flag not signed binaries (Signed/SignatureStatus syntax) and making the sigma rule be able to detect it.

We can also rewrite the sigma rule to check for where the files are located if they are in “%TEMP%, “%APPDATA%” or “%LOCALAPPDATA%”. Note these paths do not require administrator privileges to write by default.